TASK 4B – REVIEW

Requirements as set in the brief:

1. The existing code provided allows user to select a flight route and show the trends for passenger numbers over time

2. Identify trends and patterns over time for:

a. AM and PM flights for two selected routes

b. The departure airport which has the largest number of passengers over time

3. The solution must be secure

4. The solution must be easy to use

5. The information must be displayed in a meaningful way

6. Make use of appropriate textual, numerical, and graphical output in a way that would be relevant to the end user

Each requirement is listed below, and the outcomes will be analysed and discussed providing examples to demonstrate.

The existing code provided allows user to select a flight route and show the trends for passenger numbers over time.

```
print("########### Please select destination airport #########
print("### 1. Dublin (DUB)")
print("### 2. Edinburgh (EDI)")
print("### 3. Glasgow (GLA)")
print("### 4. London Heathrow (LHR)")
print("### 5. London Luton (LTN)")
print("### 6. Manchester (MAN)")



try:
    choice = int(input('Enter your number selection here: '))
```

The solution uses route menu will provide the user with a list of options and will run the corresponding functions based on the input from user.

The menu's output is meaningful and easy to use as to select, the user only needs to enter a corresponding number for each option.

The code handles user errors such as entering a number out of menu range or input that is not a number or entering nothing at all well.

```
########### Please select departure airport #############
### 1. Dublin (DUB)
### 2. Edinburgh (EDI)
### 3. Glasgow (GLA)
### 4. London Heathrow (LHR)
### 5. London Luton (LTN)
### 6. Manchester (MAN)
Enter your number selection here: 1
```

The main loop will continue running until the input the user gives is valid.

```
##################################################
#### Welcome to Elanp Air Flight Data system ####
##################################################

########### Please select departure airport #############
### 1. Dublin (DUB)
### 2. Edinburgh (EDI)
### 3. Glasgow (GLA)
### 4. London Heathrow (LHR)
### 5. London Luton (LTN)
### 6. Manchester (MAN)
Enter your number selection here: a
Sorry, you did not enter a valid option
########### Please select departure airport #############
### 1. Dublin (DUB)
### 2. Edinburgh (EDI)
### 3. Glasgow (GLA)
### 4. London Heathrow (LHR)
### 5. London Luton (LTN)
### 6. Manchester (MAN)
Enter your number selection here:
```

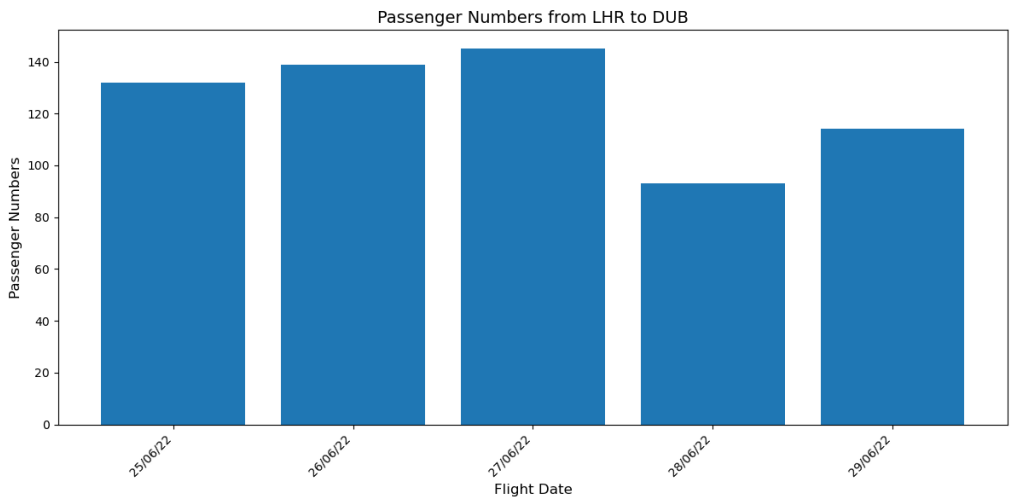
However in the provided solution the values are hardcoded therefore it does not allow for flexible route adjustment, it could be improved to fetch the destinations and departures from the file instead of being hardcoded therefore allowing for greater flexibility, as if the routes change they will automatically update from the

file instead of having to go back and change the code therefore increasing the maintainability of the program

The solution also allows users to view the data as a bar graph allowing for easy analysing of trends for each route in each period, graphs can also be saved as PNG files to allow for future analysis.

```python
def bar(data_to_plot):
    global extracted_data

    dates = data_to_plot.columns.tolist()
    passenger_numbers = data_to_plot.iloc[0].tolist()
    plt.figure(figsize=(12, 6))
    plt.bar(dates, passenger_numbers)
    plt.xlabel('Flight Date', fontsize=12)
    plt.ylabel('Passenger Numbers', fontsize=12)
    plt.title(f'Passenger Numbers from {dep_choice} to {dest_choice}', fontsize=14)
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```



Alternatively different ways to display the data are included for optimal user experience, the solution also allows the user to view the raw data as a data fame if they wish to view raw figures.

	25/06/22	26/06/22	27/06/22	28/06/22	29/06/22
80	132	139	145	93	114
81	111	106	117	129	170
82	117	100	141	146	176
83	135	155	133	126	158
84	145	123	146	111	130
85	163	122	179	175	105
86	145	175	116	91	147
87	158	161	113	175	166
88	149	139	107	85	108
89	113	122	133	169	90
90	133	146	107	164	177
91	162	156	104	143	101
92	171	153	156	88	143
93	91	176	101	178	109

Identify trends and patterns over time for AM and PM flights for two selected routes

With the dataset and solution provide it is not possible to implement the addition of trend identification for AM and PM flights. This is because the data given only shows flights separated based from day not time.

With the introduction of new data, it would be possible to implement this trend identification easily as the code is well structured and maintainable.

The departure airport which has the largest number of passengers over time

The solution created allows for a range of different analytical functions such as total passengers, busiest and least busiest days and overall passenger count.

```python
def passTotal():
    print("\n1. TOTAL PASSENGERS PER DAY:")
    daily_totals = data_frame.sum(axis=0)  # sum across rows
    for date, total in daily_totals.items():
        print(f"   {date}: {total} passengers")
    analysisMenu()

def bestWorstDays():
    global daily_totals
    print("\n2. BEST & WORST DAYS:")
    daily_totals = data_frame.sum()

    best_day = daily_totals.idxmax()
    best_value = daily_totals.max()

    worst_day = daily_totals.idxmin()
    worst_value = daily_totals.min()

    print(f"   Best day: {best_day} ({best_value} passengers)")
    print(f"   Worst day: {worst_day} ({worst_value} passengers)")
    analysisMenu()

def overallAnalysis():
    print("\n3. OVERALL:")
    daily_totals = data_frame.sum()
    all_total = daily_totals.sum()
    print(f"   All passengers: {all_total}")
    print(f"   Average per day: {all_total/len(daily_totals):.0f}")
    analysisMenu()
```

```
1. TOTAL PASSENGERS PER DAY:
   25/06/22: 1925 passengers
```

```
BEST & WORST DAYS:
Best day: 26/06/22 (1973 passengers)
Worst day: 27/06/22 (1798 passengers)
```

The solution does not fully meet the requirement as it does not separate the total passenger count based on the airport.

The solution must be secure

The solution minimally uses global variables with only one being used throughout the program in to keep it running. This could be updated using a wrapper function instead of a loop to improve security of the solution.

Only one global variable is used throughout the program. When data is needed, instead of using global variables data is passed into functions using parameters. (as shown in screenshot below)

```
def get_destination(depart):
```

Error messages outputted to the user do not display any program data and instead output a clear general description of the error that occurred and what caused it. For example: Value error, must be an integer!

```
except ValueError:
    print("Sorry, you did not enter a valid number")
```

```
3. OVERALL:
   All passengers: 9463
   Average per day: 1893
```

The solution must be easy to use

The solution has been developed with the user in mind. The messages clearly communicate to the user at every step of the process.

```
elif choice in range(1,7):
    print("Departure choice accepted!")
    return choice
else:
    print("Sorry, you did not enter a valid option")
```

```
Enter your number selection here: 1
Departure choice accepted!
```

```
########### Please enter the number of previous days of data you wish to see #############
Enter your number selection here: 5
########### You have chosen to see data for the last 5 days #############
You have selected departure from: LHR
You have selected destination as: DUB
```

```
print(f"########### You have chosen to see data for the last {days} days #############")
return days
```

Each menu provides a menu selection which requires the user to enter an integer only instead of writing text to make it easy to use and avoid typing errors and possible capitalization issues.

```
def dataMenu():
    dataOption = int(input("""### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
Enter: """))
```

```
### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
```

```
#################################################
#### Welcome to Elanp Air Flight Data system ####
#################################################

########### Please select departure airport #############
### 1. Dublin (DUB)
### 2. Edinburgh (EDI)
### 3. Glasgow (GLA)
### 4. London Heathrow (LHR)
### 5. London Luton (LTN)
### 6. Manchester (MAN)
Enter your number selection here: 4
Departure choice accepted!
########### Please select destination airport #############
### 1. Dublin (DUB)
### 2. Edinburgh (EDI)
### 3. Glasgow (GLA)
### 4. London Heathrow (LHR)
### 5. London Luton (LTN)
### 6. Manchester (MAN)
Enter your number selection here: 1
Departure choice accepted!
########### Please enter the number of previous days of data you wish to see #############
Enter your number selection here: 5
########### You have chosen to see data for the last 5 days #############
You have selected departure from: LHR
You have selected destination as: DUB
### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
Enter: 2
Generating barchart
### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
Enter: 1
    25/06/22  26/06/22  27/06/22  28/06/22  29/06/22
80       132       139       145        93       114
81       111       106       117       129       170
82       117       100       141       146       176
83       135       155       133       126       158
84       145       123       146       111       130
85       163       122       179       175       105
86       145       175       116        91       147
87       158       161       113       175       166
88       149       139       107        85       108
89       113       122       133       169        90
90       133       146       107       164       177
91       162       156       104       143       101
92       171       153       156        88       143
93        91       176       101       178       109
### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
```

The solution provides the user with a number of easy to navigate menus clearly displaying all options and the corresponding input

The information must be displayed in a meaningful way

The user can choose to output data as ether CSV raw data, bar chart, further analyse data or get table metadata.

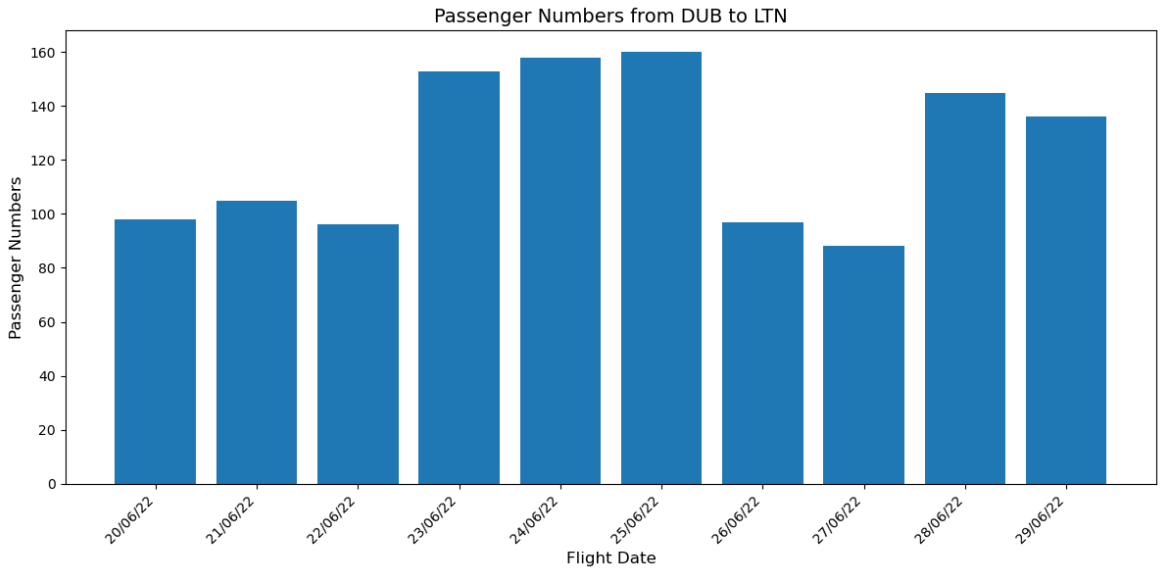
```
### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
```

```
Enter: 1
    20/06/22  21/06/22  22/06/22  23/06/22  24/06/22  25/06/22  26/06/22  27/06/22  28/06/22  29/06/22
6         98       105        96       153       158       160        97        88       145       136
7         94       108       152       134       120       114       132        88       155       102
8        108       152       128        95        89       132       113       102       120        91
9        126       151       150       145       144       103       124       137       157       112
10       125       124       103       129       133        91       120       109       116       159
11        85       154        93       130       143        91        99        97        85       155
12       111       127       113       157       148       134       111       136       124        98
13       121       160       114       149       100        88       155       151       124       154
14        96       114       132       123        98       103       123       136       123       115
15        96       107       157       141       114       130       156       100       131       118
16        95       124       117       113       131        94       130       112       140       118
17       128       149        96       116       101       149       159        93       159       121
18       101       128       156       142       146       115       153        90       145       146
### - Menu - ###
1 - View passenger data (Raw Data)
2 - View passenger data (Bar Chart)
3 - View passenger data (Analysed Breakdown)
4 - View table metadata
5 - Back to main menu
Enter:
```

```python
def bar(data_to_plot):
    global extracted_data

    dates = data_to_plot.columns.tolist()
    passenger_numbers = data_to_plot.iloc[0].tolist()
    plt.figure(figsize=(12, 6))
    plt.bar(dates, passenger_numbers)
    plt.xlabel('Flight Date', fontsize=12)
    plt.ylabel('Passenger Numbers', fontsize=12)
    plt.title(f'Passenger Numbers from {dep_choice} to {dest_choice}', fontsize=14)
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

The above code generates a bar chart using the values extracted from the table in order to provide the user with a clear, easy to read and meaningul way to view the data.



This is enhanced by the use of vibrant colouring, a clear title for the graph detailing what it is shown and clear easy to read lables on the x and y axis allowing for quick visual representation of data.

Make use of appropriate textual, numerical, and graphical output in a way that would be relevant to the end user

The solution provides information in two different easy to understand ways:

- Graphs
  - Display of plots to provide and intuitive and east to read way to visualise data making complex data sets easily understandable to carry out analysis of different routes and times. They are clearly lab led with meaningful names.
- Plain text output in table form
  - Solution also provides the option for plain text output in a table form. It is clearly displayed with meaningful row and collum names alongside row numbers.
- Clear and accurate messaging
  - Error messages display short meaningful output, clearly informing the user of the nature and cause of the errors.

Recommended Future Improvements:

1 - Hardcoded Values

One improvement that could be made is in the main aspect departure and destination menu, the values are hardcoded therefore it does not allow for flexible route adjustment, it could be improved to fetch the destinations and departures from the file instead of being hardcoded therefore allowing for greater flexibility, as if the routes change they will automatically update from the file instead of having to go back and change the code therefore increasing the maintainability of the program

2 - AM / PM Trend Identification

Another improvement that could be made to the program is it mentions in the client brief that the program should identify patterns in AM and PM flights, although the developed program includes many analytical features it does not meet these criteria as it is not able to separate the data into AM and PM flights.

3 - Data Storage Method

Another improvement that could be made is to the data storage function of this system, currently the flight data is stored in a CSV file, while this has high cross combability there are more efficient, secure and organised formats to store this data in such as using an organised database file.

4 - GUI

One more change that could be added to the system in order to improve is through the use of a Graphical User Interface (GUI) to allow users to easily navigate throughout the system menus and features more easily, this is because currently this system utilizes a Command Line Interface (CLI) which is not very user focused and may be more complicated to navigate.